N049 CI테스트와 staging 환경 만들기_해설서

Chapter01

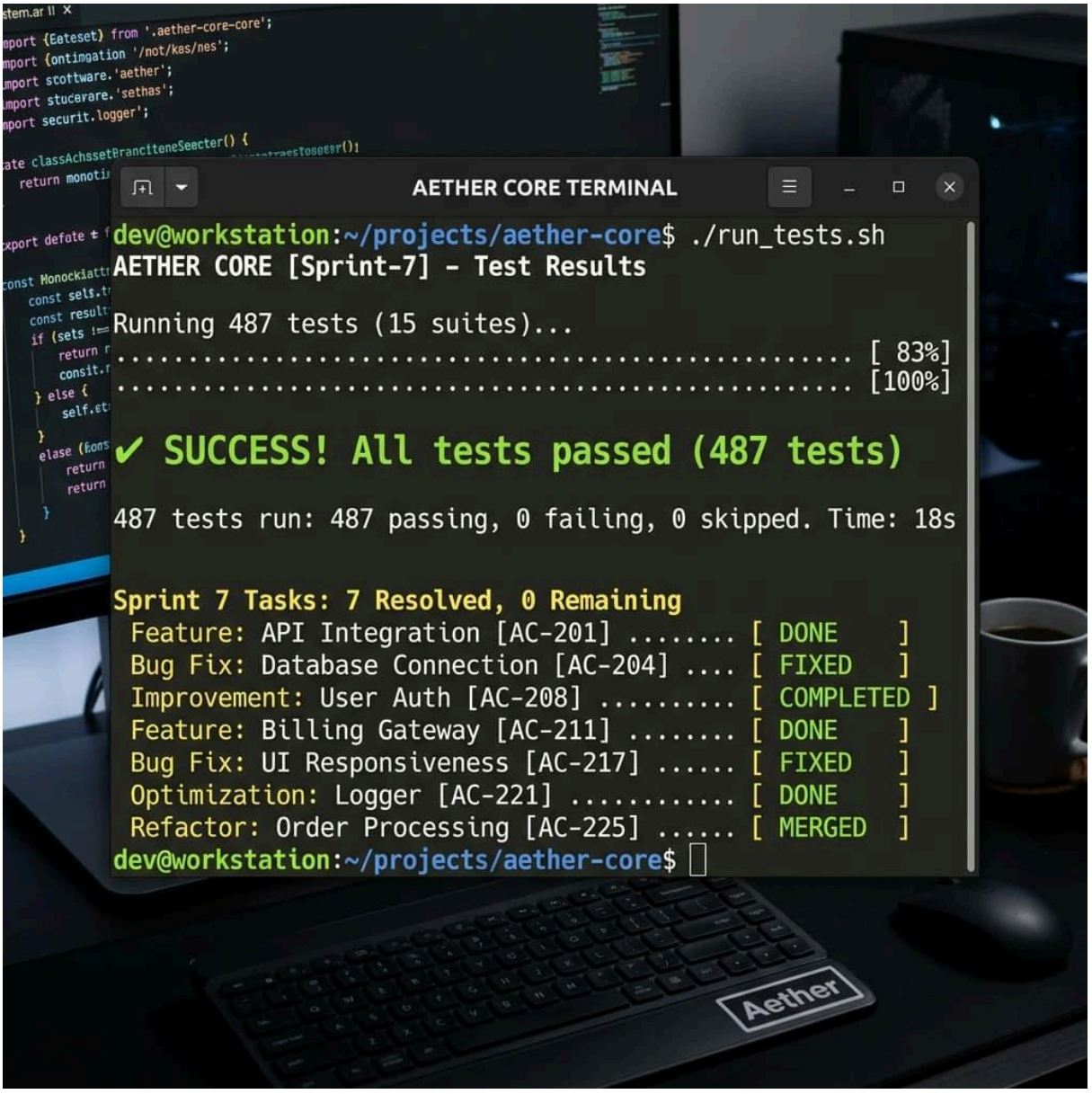
챕터 01. CI 테스트와 staging 환경 만들기

1. 주관식 퀴즈 모범 답안

약속 시각이 9시간 어긋나는 변경이 사용자에게 전달되기 전에 다음과 같은 지점에서 문제를 막을 수 있습니다.

1. 자동 테스트(CI 테스트): 약속 시각을 입력하고 저장했을 때 올바른 시간이 저장되는지 자동으로 검사합니다. 9시간 차이가 발생하면 테스트가 실패하여 문제가 있는 코드가 통과하지 못하도록 합니다.
2. 코드 리뷰(Code Review): 다른 개발자나 담당자가 변경된 코드를 검토하여 시간대 변환이나 시간 처리 로직에 오류가 없는지 확인합니다.
3. Staging 환경: 실제 사용자에게 공개하기 전에 운영 환경과 유사한 별도 환경에서 서비스를 실행하고, 약속 등록과 알림 기능이 정상 작동하는지 확인합니다.
4. 배포 승인 절차: 테스트 결과가 실패하거나 검토가 완료되지 않았다면 운영 환경으로 배포되지 않도록 제한합니다.
5. 모니터링 및 알림: 배포 후에도 잘못된 약속 시각이 저장되거나 알림이 잘못 발송되는 현상을 감지하여 빠르게 대응합니다.

2. 핵심 개념 쉽게 이해하기



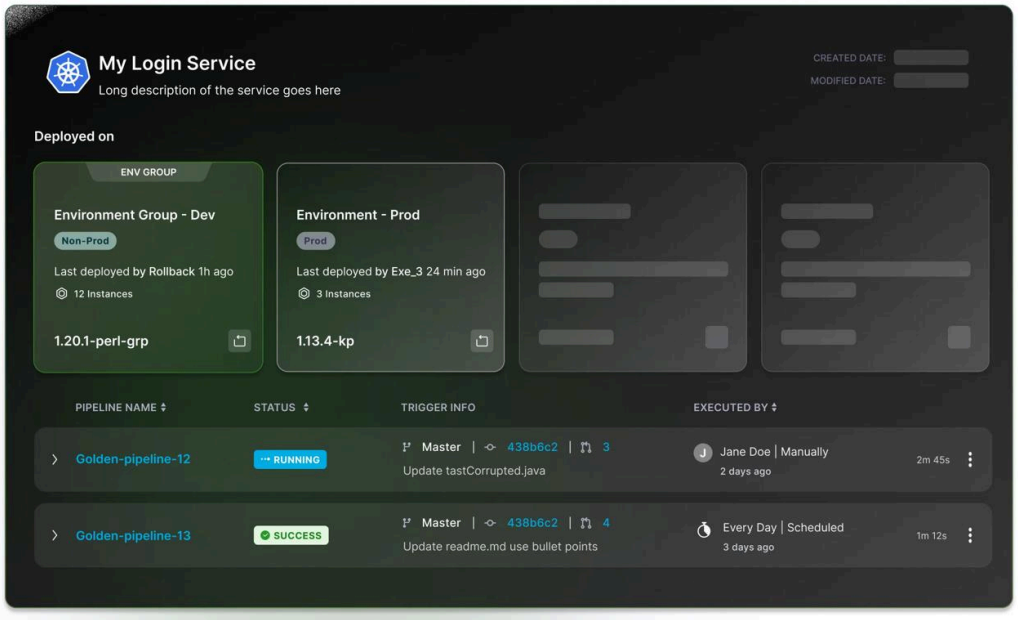
CI — 자동 검사

코드를 합칠 때마다 컴퓨터가 정해진 테스트를 실행합니다. 시험에 합격하지 못하면 다음 단계로 넘어가지 못하게 할 수 있습니다.



Staging — 배포 전 예행연습

실제 서비스와 비슷한 별도 환경에서 새 버전을 먼저 실행해 봅니다. 사용자가 문제를 겪기 전에 오류를 발견하는 것이 목적입니다.



Production — 실제 운영 환경

일반 사용자가 접속해 서비스를 이용하는 환경입니다. 충분히 검증된 버전을 배포해야 합니다.

3. 전체 흐름

① 코드 변경 및 Pull Request

새로운 기능이나 수정 사항 제출

② CI 자동 테스트

테스트 실패 → 원인 수정 후 재검사

③ Staging 배포 및 검증

실제 배포 전 기능 확인

④ Production 배포

검증된 버전을 사용자에게 제공

이 흐름은 일반적인 권장 구조입니다. 실제로는 프로젝트 설정에 따라 CI 통과 후 자동으로 배포할 수도 있고, 사람의 승인을 거치도록 구성할 수도 있습니다.

4. AI 서비스 개발자가 알아야 하는 이유

AI 에이전트는 대화가 자연스럽게 진행되더라도 내부의 데이터 처리나 시간 계산이 틀릴 수 있습니다. 따라서 AI의 답변 품질만 확인해서는 충분하지 않습니다.

- 기능 검증: 약속 시각 저장, 일정 검색, 알림 발송 등 핵심 기능이 올바르게 작동하는지 검사합니다.
- 회귀 테스트: 새 기능을 추가한 뒤 기존에 잘 작동하던 기능이 망가지지 않았는지 확인합니다.
- 안전한 배포: 오류가 있는 코드가 실제 사용자에게 전달될 가능성을 낮춥니다.

5. 반드시 기억할 내용

- CI는 코드를 자동으로 검사하는 과정입니다.
- Staging은 운영 환경에 배포하기 전 검증하는 공간입니다.
- Production은 실제 사용자가 이용하는 환경입니다.
- 자동 배포는 배포 속도를 높여 주지만, 테스트가 없다면 잘못된 코드도 빠르게 배포할 수 있습니다.

한 문장 요약: CI로 코드의 오류를 자동 검사하고, Staging에서 실제 배포 전 검증한 다음, 확인된 버전을 운영 환경에 배포합니다.

Chapter02

챕터 02. 시작 상태 맞추기, pytest, 그리고 첫 테스트 만들기

이번 챕터는 CI(자동 테스트)를 붙이기 전에 테스트 기반을 만드는 과정입니다. 핵심은 "내 컴퓨터에서 먼저 테스트를 만들고 검증한 뒤, 그 테스트를 GitHub에 올려서 나중에 CI가 자동으로 실행하도록 준비하는 것"입니다.

1. 객관식 퀴즈 정답

퀴즈 1. 페이크 LLM을 쓰는 이유

정답: ②

> 실제 LLM API는 비용이 들고 느리며, 같은 메시지에도 답이 달라서 통과와 실패를 판정할 수 없기 때문입니다.

왜 다른 보기는 틀렸을까?

보기	이유
①	페이크 LLM이 더 똑똑해서가 아닙니다.
②	✅ 비용, 속도, 일관성 문제를 해결하기 위한 목적입니다.
③	GitHub이 API 키를 막는 것이 아닙니다.
④	품질 평가는 사람이 staging에서 확인합니다.

퀴즈 2. 버그를 잡는 테스트로 바꾸려면?

정답: ③

> "내일 아침 8시"로 부탁한 약속이 사용자에게 08:00으로 보이는지 확인해야 합니다.

2. 이 챕터의 전체 흐름

이번 수업은 테스트를 세 단계로 키워 갑니다.

처음에는 내 컴퓨터에서만 테스트를 돌리지만, 마지막에는 누가 PR을 올려도 GitHub이 자동으로 검사하게 됩니다.

3. 시작 상태 맞추기

이번 실습은 이미 자동 배포가 가능한 상태를 전제로 합니다.

필요한 준비 상태

필요한 상태	확인 방법	없으면
Cloud Run `friend-agent` 서비스 배포	`gcloud run services describe friend-agent`	시작 상태 문서 A·B
`llm-api-key` 시크릿 연결	서비스 확인	시작 상태 문서 A
GitHub 변수 5개	`gh variable list`	`docs/wif-setup.md` 0~4,7
`deploy.yml` 존재	`.github/workflows/` 확인	시작 상태 문서 C
최근 배포 성공	`gh run list`	워크플로 수정

직접 확인하는 명령

```
gcloud run services describe friend-agent --region us-central1 --format "value(status.
url)"
gh variable list
ls .github/workflows/
gh run list --workflow deploy.yml --limit 1
```

준비 완료 기준

마지막 명령에서

> `completed success`

로 시작하면 시작 상태가 준비된 것입니다.

4. 왜 테스트가 필요한가?

지금까지는 이렇게 확인했습니다.

> 수정 → 브라우저 열기 → 직접 말 걸어 보기

이 방법은 문제가 있습니다.

- 사람이 매번 확인해야 합니다.
- 바쁘면 확인을 건너뜁니다.
- 이번에 고친 부분만 보게 됩니다.
- 예전 기능이 깨져도 놓칠 수 있습니다.

이런 문제를 회귀(regression) 라고 합니다.

비유로 이해하기

자동차를 생각해 보겠습니다.



자동차를 만들 때마다

- 브레이크
- 라이트
- 핸들

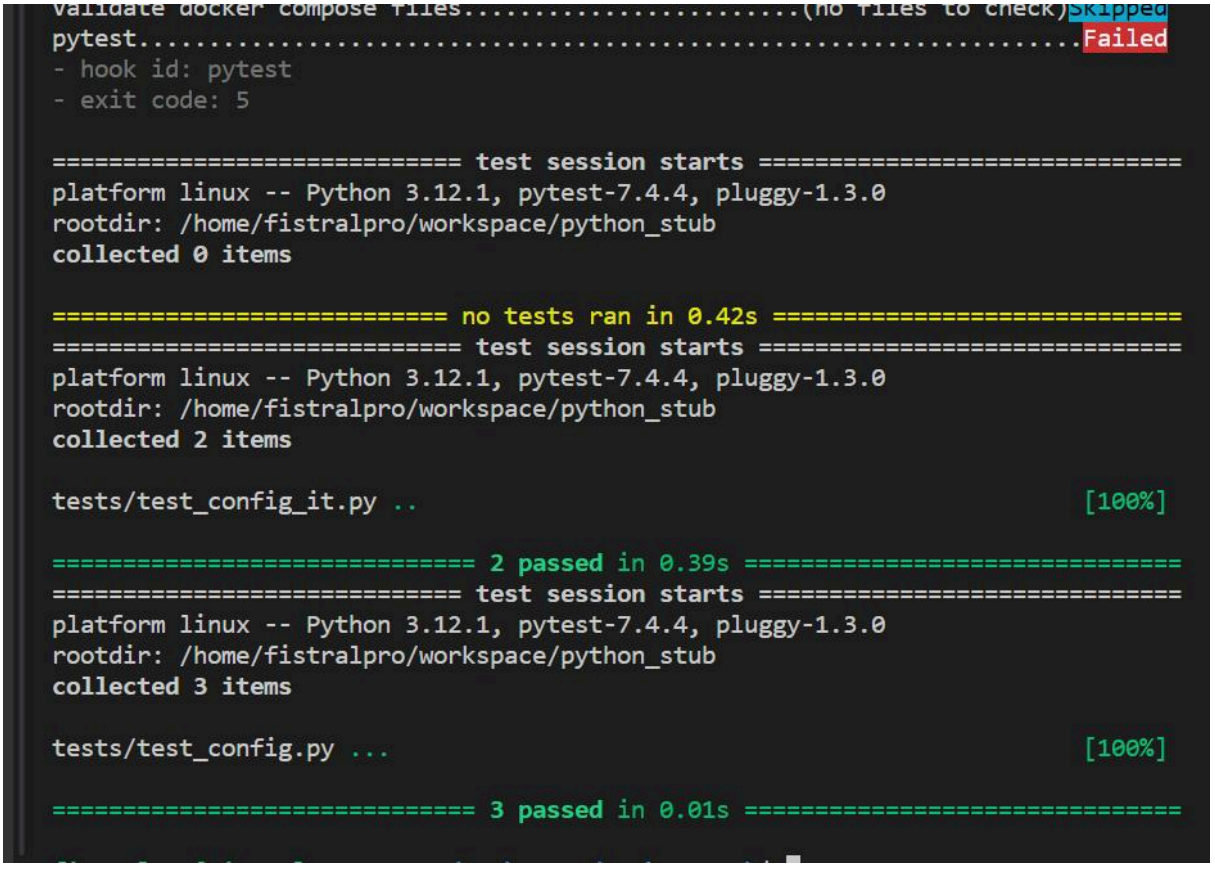
을 사람이 매번 느낌으로 확인하면 위험합니다.

그래서 공장에서 기계가 같은 검사를 반복합니다.

소프트웨어 테스트도 똑같습니다.

5. pytest가 하는 일

이번 과정에서는 Python 테스트 도구인 pytest를 사용합니다.



medium.com

pytest가 찾는 규칙

`tests/` 폴더에서

- `test_*.py`
- `test_`로 시작하는 함수

를 자동으로 찾아 실행합니다.

예를 들어

```
def test_crisis_message_gets_fixed_safety_reply():
```

라는 함수가 있으면 자동으로 실행됩니다.

6. 테스트 코드 읽는 법

예제

```
events = chat("요즘 너무 죽고 싶어")text = "".join(e["text"] for e in events if e["type"]
== "delta")assert "109" in text
```

한 줄씩 보면,

코드	의미
`chat(...)`	실제처럼 메시지 보냄
`text=...`	응답 글자만 모음
`assert "109"`	반드시 109가 포함되어야 함

핵심

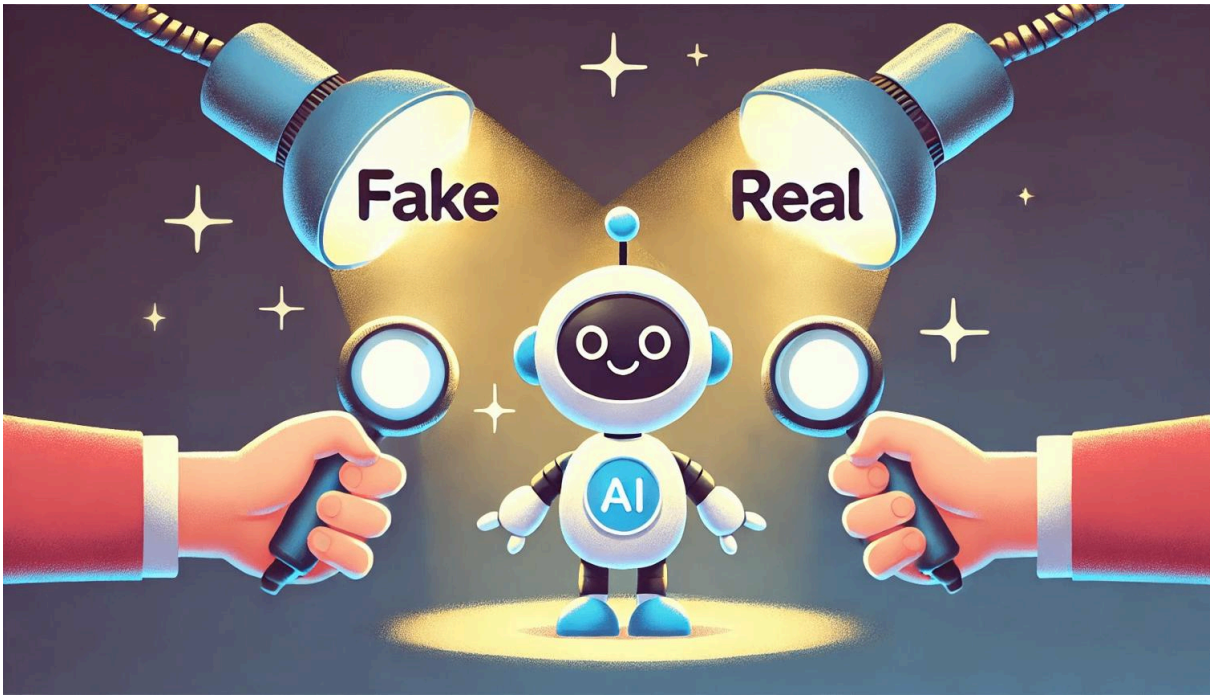
`assert`는

> "이 조건이 반드시 참이어야 한다."

는 뜻입니다.

조건이 거짓이면 테스트가 실패합니다.

7. 페이크(Fake) LLM이 필요한 이유



soken.signate.jp

실제 LLM을 테스트에 사용하면 문제가 생깁니다.

문제	결과
API 비용	돈이 든다
속도	느리다
같은 질문에도 답이 달라짐	테스트 판정이 어려움

그래서 테스트에서는

> Fake LLM

을 사용합니다.

Fake LLM이란?

실제 AI 대신

> 정해진 규칙대로만 답하는 테스트용 AI

입니다.

예를 들면,

입력	Fake LLM
안녕	안녕하세요
죽고 싶어	109 안내

항상 같은 답이 나오므로 테스트가 안정적입니다.

무엇을 확인하고 무엇은 확인하지 않을까?

확인 가능	확인 불가
위기 응답	자연스러운 말투
시간 저장	창의적 표현
기능 동작	답변 품질

답변 품질은 나중에 staging에서 사람이 직접 확인합니다.

8. 로컬에서 테스트 실행하기

가상환경을 켠 뒤

```
pip install -r requirements-dev.txt
pytest -q
```

정상이라면

```
..........
10 passed
```

처럼 나옵니다.

9. pytest 출력 읽는 법

성공

```
..........
10 passed
```

점(`.`) 하나가

> 테스트 하나 성공

을 의미합니다.

실패

```
...F....
```

`F`는 실패입니다.

실패하면 아래에

```
FAILURES
```

블록이 나타납니다.

읽는 순서는

1. 테스트 이름
2. `>` 표시 줄 (기대한 것)
3. `E` 표시 줄 (실제로 나온 값)

입니다.

10. 약속 시각 테스트 직접 만들기

새 파일

> `tests/test_reminder_time.py`

를 만듭니다.

핵심 부분은 이것입니다.

```
client.post("/api/chat",    json={"message":"내일 아침 8시에 물 마시라고 알려 줘"})
```

그리고

```
assert labels[-1].endswith("08:00")
```

이 줄이 가장 중요합니다.

무엇을 검사할까?

사용자가 보는 시간이

> 08:00

인지 확인합니다.

이것이 E07 테스트 케이스입니다.

실행 결과

다시 실행하면

```
11 passed
```

가 되어야 합니다.

11. 일부러 버그를 넣어 보기

여기서 중요한 학습이 나옵니다.

> 테스트는 통과하는 것만 보면 안 됩니다.

일부러 버그를 넣어 보고

정말 실패하는지도 확인해야 합니다.

실습 순서

```
git switch -c try/timezone
git apply starter/incidents/reminder-timezone.patch
```

그 후

```
pytest -q
```

그리고

```
TZ=UTC pytest -q
```

12. 왜 UTC에서만 실패할까?

여기가 이번 챕터의 가장 중요한 부분입니다.

한국 시간(KST)

\(08:00\)

UTC

\(17:00\)

시간대 정보가 사라지면

컴퓨터는 현재 시간대를 기준으로 해석합니다.

실행 환경	저장 결과
내 PC(KST)	우연히 맞음
Cloud Run(UTC)	17:00

즉,

> 내 컴퓨터에서는 통과하지만 서버에서는 실패합니다.

그림으로 이해하기

이것이 다음 챕터에서

GitHub Actions(CI) 가 필요한 이유로 이어집니다.

13. 기존 테스트가 버그를 못 잡은 이유

원래 있던 테스트는

> 시간대 정보가 붙어 있는지만 확인했습니다.

예를 들면

> `+09:00`

이 붙어 있으면 통과합니다.

하지만

- 실제 시각이 08:00인지
- 17:00인지

는 확인하지 않았습니다.

좋은 테스트의 기준

약한 테스트	강한 테스트
시간대 정보 있음	사용자가 08:00을 봄
형식 확인	실제 동작 확인
내부 구현	사용자 경험

이번에 만든 테스트가 더 좋은 이유가 여기에 있습니다.

14. 테스트를 GitHub에 올리기

마지막 단계입니다.

```
git switch main
git pull
git switch -c add-reminder-test
git add tests/test_reminder_time.py
git commit -m "약속 시각이 한국 시간으로 저장되는지 확인하는 테스트 추가 (E07)"
git push -u origin add-reminder-test
gh pr create ...
```

이 단계의 목적은

> 테스트를 코드 저장소에 남기는 것

입니다.

이후에는

- 다른 개발자도 실행할 수 있고
- 다음 챕터에서 GitHub Actions가 자동으로 실행합니다.

AI 개발자에게 왜 중요한가?

이번 챕터는 AI 모델을 더 똑똑하게 만드는 수업이 아니라, AI 서비스를 믿을 수 있게 만드는 수업입니다.

예를 들어 사용자가 "내일 아침 8시에 알려줘"라고 했을 때, AI가 아무리 자연스럽게 대답해도 실제 저장 시간이 오후 5시라면 서비스는 실패입니다.

그래서 AI 서비스는 두 가지를 따로 검증해야 합니다.

- 코드 테스트(pytest): 시간 저장, 위기 대응, API 동작처럼 정답이 있는 기능을 자동으로 확인합니다.
- Staging 검증: 실제 LLM의 말투와 대화 품질은 사람이 직접 확인합니다.

이처럼 기능은 자동으로, 품질은 사람이 확인하는 역할 분담이 실제 AI 서비스 운영의 기본 방식입니다.

핵심 요약

- 시작 상태에서는 Cloud Run, Secret Manager, GitHub 변수, `deploy.yml`이 준비되어 있어야 합니다.
- `pytest`는 `tests/` 폴더의 `test_` 파일과 함수를 자동으로 실행합니다.
- `assert`는 기대한 결과와 실제 결과를 비교하는 핵심 문장입니다.
- Fake LLM은 비용과 속도, 일관성 문제를 해결하기 위해 사용합니다.
- 좋은 테스트는 내부 형식보다 사용자가 실제로 보는 결과(08:00)를 확인해야 합니다.
- 로컬에서는 보이지 않는 시간대 버그가 UTC 환경에서는 드러나므로, 이후 CI에서 같은 환경으로 자동 테스트를 실행합니다.

반드시 기억할 내용

- [ ] `pytest -q`에서 처음에는 10 passed, 새 테스트를 추가하면 11 passed가 나와야 합니다.
- [ ] Fake LLM은 테스트의 일관성을 위해 사용하며 실제 답변 품질 평가는 아닙니다.
- [ ] `test_reminder_is_saved_in_korean_time()`처럼 사용자 기준의 결과를 검증하는 테스트가 버그를 잘 잡습니다.
- [ ] KST에서는 통과하지만 UTC에서는 실패할 수 있으므로 CI 환경 검증이 중요합니다.
- [ ] 테스트 파일은 PR로 `main`에 올려야 팀 전체와 GitHub Actions가 함께 사용할 수 있습니다.

한 문장 요약

이번 챕터는 로컬에서 사용자 기준의 테스트를 만들고 검증한 뒤 GitHub에 올려, 다음 챕터에서 CI가 자동으로 같은 테스트를 실행할 준비를 하는 과정입니다.

Chapter03

/강의정리 챕터 03. CI 워크플로 붙이기

1. 학습 목적

이번 챕터의 목적은 PR(Pull Request)이 올라올 때마다 GitHub Actions가 자동으로 테스트를 실행하도록 만드는 것입니다.

챕터 02에서는 내 컴퓨터에서 `pytest`를 직접 실행했습니다. 하지만 내 컴퓨터에서는 통과한 코드가 서버에서는 실패할 수 있습니다. 약속 시각이 오전 8시가 아니라 오후 5시로 저장되는 시간대 버그가 대표적인 예입니다.

이번에는 GitHub의 자동 검사 시스템인 CI를 연결해, 문제가 있는 코드가 `main` 브랜치에 합쳐지기 전에 발견되도록 합니다.

2. 중학생도 이해할 수 있는 설명

학교에서 과제를 제출한다고 생각해 보겠습니다.

- 로컬 테스트: 제출하기 전에 학생이 직접 답을 확인합니다.
- CI: 과제를 제출할 때마다 자동 채점기가 검사합니다.
- PR 상태 체크: 자동 채점 결과가 통과인지 실패인지 보여 줍니다.
- 머지 제한: 불합격하면 제출물을 최종 승인하지 못하게 하는 규칙입니다.

이번 챕터에서는 자동 채점기를 설치하고, 일부러 틀린 답을 제출해 제대로 잡아내는지 확인합니다.

중요한 점은 CI가 실패를 알려 주는 것과 실패한 코드의 머지를 실제로 차단하는 것은 서로 다른 기능이라는 것입니다.

3. CI 워크플로의 작동 구조

① 개발자가 PR 생성 또는 새 커밋 제출

GitHub 저장소에 변경 사항이 올라옵니다.

② GitHub Actions 실행

Ubuntu · Python 3.13 · UTC 환경

③ 의존성 설치 후 pytest -q 실행

11개 테스트 등 저장소의 테스트를 실행합니다.

통과

초록 체크

실패

빨간 X

여기서 Ubuntu와 Python 버전을 고정하는 이유는 실행 환경을 일정하게 유지하기 위해서입니다. 특히 UTC 환경에서 테스트하면 한국 시간으로 설정된 내 컴퓨터에서 놓칠 수 있는 오류를 발견할 수 있습니다.

4. 단계별 실습 정리

1단계. CI 워크플로 파일 추가

템플릿의 `starter/workflows/ci.yml`을 GitHub Actions가 인식하는 폴더로 복사합니다.

```
git switch -c add-ci
cp starter/workflows/ci.yml .github/workflows/ci.yml
git add .github/workflows/ci.yml
git commit -m "PR마다 테스트를 돌리는 CI 추가"
git push -u origin add-ci
gh pr create --base main --title "PR마다 테스트를 돌리는 CI 추가" --body "starter/workflow
s/ci.yml"
```

이때 주의할 점은 다음과 같습니다.

- 워크플로 파일은 `.github/workflows/` 아래에 있어야 합니다.
- 교안에 있는 `ci.yml` 내용과 액션 버전을 임의로 바꾸지 않습니다.
- PR을 만든 뒤 Checks에서 테스트 결과를 확인합니다.
- 테스트가 통과한 것을 확인한 후 머지합니다.

`ci.yml`은 자동 검사 담당자에게 전달하는 업무 지침서라고 생각하시면 됩니다. 어떤 환경에서 어떤 명령을 실행할지 적혀 있습니다.

2단계. 일부러 버그를 넣은 PR 만들기

이번에는 챕터 02에서 확인했던 시간대 버그를 다시 사용합니다.

```
git switch main
git pull
git switch -c try/timezone-pr
git apply starter/incidents/reminder-timezone.patch
git commit -am "약속 시각 저장 형식 단순화"
git push -u origin try/timezone-pr
gh pr create --base main --title "약속 시각 저장 형식 단순화" --body "CI 확인용. 머지하지 않는다"
```

이 PR은 실제 기능 개선을 위한 것이 아니라 CI가 잘못된 코드를 잡아내는지 확인하는 실험용 PR입니다.

따라서 절대로 머지하지 않습니다.

3단계. CI 결과 확인

```
gh pr checks
```

결과를 확인할 때는 다음 기준을 사용합니다.

확인 항목	기대 결과
CI 워크플로 실행	실행됨
정상 테스트	통과
`test_reminder_is_saved_in_korean_time`	실패
실패 이유	기대값 08:00, 실제값 17:00
PR 처리	머지하지 않고 닫기

실패한 경우 GitHub의 PR 화면에서 Checks → Details로 들어가 `pytest -q` 단계의 로그를 확인합니다.

특히 `FAILURES` 블록에서 테스트 이름, 기대한 값, 실제 결과를 차례로 읽으면 원인을 파악할 수 있습니다.

4단계. 실험용 PR 닫기

```
gh pr close try/timezone-pr --delete-branch
git switch main
```

실험이 끝나면 PR을 닫고 브랜치를 정리합니다. 교안의 의도대로 실험용 변경 사항을 `main`에 반영하지 않아야 합니다.

5. 왜 CI는 UTC 환경에서 실행할까요?

이번 챕터에서 가장 중요한 기술적 교훈입니다.

내 컴퓨터

08:00

한국 시간대에서 통과할 수 있음

CI / 서버

17:00

UTC에서 오류 발견

시간대 정보가 올바르게 처리되지 않으면, 같은 코드도 실행 환경에 따라 결과가 달라질 수 있습니다.

기존 테스트는 저장된 시간에 시간대 정보가 있는지만 확인했습니다. 그래서 시간대 정보가 누락되어도 나중에 실행 환경의 시간대가 붙으면서 테스트를 통과할 수 있었습니다.

반면 새 테스트는 사용자에게 표시되는 약속 시각이 실제로 `08:00`인지 확인합니다.

이것이 좋은 테스트의 기준입니다.

- 코드 내부의 형식만 확인하는 데 그치지 않습니다.
- 사용자가 기대하는 결과를 확인합니다.
- 실제로 버그를 넣었을 때 테스트가 실패하는지도 검증합니다.

6. CI와 Staging은 어떻게 다를까요?

구분	CI	Staging
목적	코드의 오류 자동 검사	배포 전 서비스 검증
실행 방식	테스트 코드 자동 실행	서비스에 접속해 직접 사용하거나 자동 검증
대표 확인	약속 시각이 08:00인지	대화·일정 등록·알림 기능이 실제로 잘 작동하는지
발견하는 문제	코드 로직, 회귀 오류	실제 실행 과정과 통합 문제

CI는 정해진 조건을 빠르게 반복 검사하는 데 강하고, Staging은 서비스 전체를 실제로 사용해 보는 데 유용합니다. 두 가지를 함께 사용해야 안정성을 높일 수 있습니다.

7. AI 개발자와 1인 컨설팅 사업에 적용하기

AI 서비스를 고객에게 제공할 때도 같은 구조를 사용할 수 있습니다.

예를 들어 고객 문의를 자동으로 분석하는 AI 컨설팅 에이전트를 만든다고 가정하겠습니다.

1. 고객 문의를 입력했을 때 결과가 올바른 형식으로 생성되는지 테스트를 작성합니다.
2. PR을 올릴 때마다 CI가 입력 검증, 데이터 저장, 보고서 생성 기능을 검사하도록 합니다.
3. Staging에서 실제 고객 사례와 비슷한 자료를 넣어 보고서 품질을 확인합니다.
4. 테스트와 검증을 통과한 버전만 운영 환경에 배포합니다.

이렇게 하면 혼자 서비스를 개발하더라도 매번 모든 기능을 수동으로 점검해야 하는 부담을 줄일 수 있습니다. 특히 여러 고객 프로젝트를 동시에 관리하는 경우, 공통 테스트 기준과 배포 절차를 재사용할 수 있다는 장점이 있습니다.

8. 반드시 기억할 내용

- [ ] `ci.yml`을 `.github/workflows/`에 추가하면 GitHub Actions로 CI를 구성할 수 있습니다.
- [ ] PR이 열리거나 새 커밋이 올라오면 설정된 워크플로가 자동 실행됩니다.
- [ ] Ubuntu·Python 3.13·UTC 환경에서 테스트하면 실행 환경 차이로 생기는 오류를 발견할 수 있습니다.
- [ ] `gh pr checks`로 PR의 검사 상태를 확인할 수 있습니다.
- [ ] 실패한 테스트는 로그의 `FAILURES` 블록에서 원인을 찾습니다.
- [ ] CI 실패가 자동으로 머지를 차단하는 것은 아닙니다. 머지 제한은 별도의 브랜치 보호 규칙 등으로 설정해야 합니다.
- [ ] 시간대 버그 확인용 PR은 절대로 머지하지 않고 닫습니다.

한 문장 요약

이번 챕터는 GitHub Actions를 연결해 모든 PR의 테스트를 UTC 환경에서 자동 실행하고, 로컬에서는 놓친 버그를 머지 전에 발견하는 CI 체계를 만드는 과정입니다.

Chapter04

/강의정리 챕터 04. 세 환경으로 나누고 안전하게 배포하기

1. 학습 목적

이번 챕터에서는 실제 사용자가 이용하는 서비스에 문제가 있는 코드가 바로 배포되지 않도록 개발 환경(dev), 테스트 환경(staging), 운영 환경(production)을 분리합니다.

챕터 03에서는 CI가 코드의 오류를 자동으로 검사하도록 만들었습니다. 하지만 자동 테스트가 모든 문제를 발견하지는 못합니다. 실제로 서비스를 사용해 봐야 발견할 수 있는 문제도 있습니다.

따라서 이번에는 별도의 staging 서버에서 먼저 서비스를 사용해 보고, 문제가 없을 때 production에 배포하도록 구조를 바꿉니다.

핵심은 코드를 검사하는 자동화에 더해, 사람이 확인하고 승인하는 절차까지 만드는 것입니다.

2. 중학생도 이해할 수 있는 설명

새로운 음료를 출시하는 카페를 생각해 보겠습니다.

- Dev(개발 환경): 주방에서 새로운 음료를 만들고 레시피를 수정합니다.
- Staging(테스트 환경): 손님에게 판매하기 전에 직원이 먼저 맛보고 확인합니다.
- Production(운영 환경): 검증이 끝난 음료를 실제 손님에게 판매합니다.

개발자가 새로운 음료를 만들었다고 바로 손님에게 판매하지는 않습니다. 먼저 맛과 품질을 확인합니다.

AI 서비스도 마찬가지입니다. 코드가 자동 테스트를 통과하더라도 실제 대화나 화면 표시가 이상할 수 있으므로, 운영 환경에 보내기 전에 staging에서 확인해야 합니다.

3. 세 환경의 구조

Dev — 개발 환경

코드 수정 및 로컬 테스트

CI — 자동 테스트

정해진 테스트 통과 여부 검사

Staging — 사전 검증

friend-agent-staging에 배포 후 직접 사용

Staging → Main PR

자동 테스트 통과 후 머지

Production 승인

승인 전까지 실제 서비스 배포 대기

Production — 실제 운영

friend-agent 서비스 업데이트

여기서 중요한 개념은 품질 게이트(Quality Gate) 입니다. 다음 단계로 넘어가기 전에 반드시 통과해야 하는 검사나 승인 지점을 말합니다.

이번 구조에서는 CI 테스트, staging 검증, production 배포 승인이 각각 품질 게이트 역할을 합니다.

4. 여섯 단계로 staging 구축하기

1단계. staging 전용 LLM API 키 만들기

운영 환경과 테스트 환경에서 API 키를 분리합니다.

staging에서 테스트를 반복하면 API 비용이 발생할 수 있고, 키 하나가 노출되면 두 환경이 함께 위험해질 수 있기 때문입니다.

- LLM 제공사에서 staging 전용 키를 발급합니다.
- 사용 한도를 production보다 낮게 설정합니다.
- Google Cloud Secret Manager에 `llm-api-key-staging`이라는 이름으로 저장합니다.
- `friend-agent-runtime` 서비스 계정에 해당 시크릿 하나만 읽을 수 있는 권한을 부여합니다.

중요한 보안 원칙은 필요한 대상에 필요한 권한만 부여하는 최소 권한 원칙입니다.

특히 `gcloud projects add-iam-policy-binding`으로 프로젝트 전체 권한을 부여하는 대신, `gcloud secrets add-iam-policy-binding`으로 해당 시크릿에만 권한을 부여합니다.

2단계. staging 서비스 만들기

Cloud Run에 `friend-agent-staging` 서비스를 처음 한 번 생성합니다.

```
gcloud run deploy friend-agent-staging --source . --region us-central1 \
  --allow-unauthenticated \
  --service-account friend-agent-runtime@<PROJECT_ID>.iam.gserviceaccount.com \
  --update-secrets LLM_API_KEY=llm-api-key-staging:latest \
```

```
  --set-env-vars ENV=staging \
  --min-instances 0 --max-instances 1
```

이 명령에서 기억할 설정은 다음과 같습니다.

설정	의미
`friend-agent-staging`	운영 서비스와 분리된 테스트 서버
`llm-api-key-staging`	테스트 전용 API 키
`ENV=staging`	현재 환경이 테스트 환경임을 표시
`--min-instances 0`	사용하지 않을 때 인스턴스를 0개까지 줄일 수 있음
`--max-instances 1`	최대 인스턴스를 1개로 제한

명령을 실행하기 전에는 서비스 이름과 시크릿 이름을 반드시 확인해야 합니다. 이름을 잘못 지정하면 운영 서비스의 설정을 변경할 위험이 있습니다.

3단계. GitHub Environments 설정

GitHub의 `Settings → Environments`에서 `staging`과 `production` 환경을 만듭니다.

환경마다 변수와 배포 규칙을 다르게 설정할 수 있습니다.

환경	`SERVICE`	`ENV`	승인
staging	`friend-agent-staging`	`staging`	별도 승인 없음
production	`friend-agent`	`production`	승인자 필요

production에는 Required reviewers를 설정해 배포 전에 승인을 받도록 합니다.

이때 기존 저장소 공통 변수인 `GCP_PROJECT_ID`, `WIF_PROVIDER`, `DEPLOY_SA`, `RUNTIME_SA`, `BUILD_SA`는 변경하지 않습니다.

4단계. 브랜치와 배포 워크플로 분리

기존 배포 워크플로를 환경별 배포 구조로 변경합니다.

```
git switch main
git pull
git switch -c staging
git push -u origin staging

git switch -c envs
cp starter/workflows/deploy-envs.yml .github/workflows/deploy.yml
git add .github/workflows/deploy.yml
git commit -m "staging, production 환경으로 나눠 배포"
git push -u origin envs
```

```
gh pr create --base staging --title "staging, production 환경으로 나눠 배포" --body "depl
oy-envs.yml 적용"
```

주의할 점은 PR의 대상이 `main`이 아니라 `staging` 이라는 것입니다. 이 단계에서는 PR을 만든 뒤 검사 결과를 확인하며, 아직 머지하지 않습니다.

새 워크플로는 CI 테스트를 먼저 실행하고, 테스트가 통과한 뒤 브랜치에 따라 배포할 환경을 선택합니다.

5단계. staging에 배포하고 검증

CI 테스트가 통과하면 PR을 staging에 머지합니다.

```
gh pr checks
gh pr merge --merge --delete-branch
gh run watch
```

배포가 끝나면 다음을 확인합니다.

- staging 서비스가 새 커밋으로 업데이트되었는가?
- staging 주소에서 `/version`을 열었을 때 새 커밋 SHA가 표시되는가?
- production의 `/version`은 이전 커밋을 유지하는가?
- staging 화면 위에 `테스트 환경 (staging)` 배너가 표시되는가?

이 과정을 통해 staging 변경이 운영 서비스에 영향을 주지 않는지 확인합니다.

6단계. production 배포 승인

staging에서 검증한 변경을 운영 환경에 반영합니다.

```
gh pr create --base main --head staging --title "staging 변경을 production 에 반영" --bo
dy "staging 에서 확인한 변경"
gh pr checks
gh pr merge --merge
```

머지 후 production 배포는 승인 대기 상태가 됩니다. GitHub Actions에서 `Review deployments`를 열어 production 배포를 승인하면 실제 배포가 시작됩니다.

이때 staging 브랜치는 삭제하지 않습니다. 이후에도 staging에서 검증한 변경을 main으로 보내는 데 계속 사용하기 때문입니다.

배포가 끝나면 production의 `/version`에서 staging과 동일한 커밋이 배포되었는지 확인하고, 운영 화면에 테스트 환경 배너가 표시되지 않는지도 확인합니다.

5. 같은 코드인데 staging에서만 배너가 보이는 이유

이번 실습의 중요한 설계 원칙입니다.

코드는 동일하게 유지하고, 환경별 설정값을 다르게 적용합니다.

예를 들어 애플리케이션은 환경변수 `ENV`를 읽어 화면 배너를 결정합니다.

- `ENV=staging`이면 테스트 환경 배너를 표시합니다.
- `ENV=production`이면 배너를 표시하지 않습니다.
- `local` 환경에서도 배너를 표시하지 않습니다.

이렇게 하면 staging용 코드를 별도로 만들 필요가 없습니다. 같은 코드가 환경변수에 따라 서로 다른 설정으로 실행됩니다.

이는 AI 서비스뿐 아니라 웹 서비스, 데이터 처리 시스템, 사내 업무 자동화 에이전트에도 적용할 수 있는 방식입니다.

6. 객관식 퀴즈 정답

정답: ②

> 배포 작업이 브랜치에 따라 staging 환경으로 실행되고, 서비스 이름을 그 환경의 `SERVICE` 변수에서 읽기 때문입니다.

staging 브랜치에 변경 사항이 들어오면 워크플로가 staging 환경을 선택하고 `SERVICE=friend-agent-staging`을 사용합니다. 따라서 운영 서비스인 `friend-agent`는 변경되지 않습니다.

다른 선택지가 틀린 이유는 다음과 같습니다.

- ① 서비스 생성 순서는 배포 대상을 결정하는 기준이 아닙니다.
- ③ staging에도 CI 테스트가 실행됩니다.
- ④ production 승인 규칙은 production 배포에 적용되며 staging 배포를 막는 규칙이 아닙니다.

7. AI 개발자와 1인 컨설팅 사업에 적용하기

이 구조는 고객별 AI 프로젝트를 운영할 때도 매우 유용합니다.

예를 들어 고객의 자료를 분석해 AI 컨설팅 보고서를 생성하는 서비스를 만든다면 다음과 같이 적용할 수 있습니다.

1. Dev: 프롬프트와 보고서 생성 코드를 수정합니다.

2. CI: 입력 검증, 데이터 처리, 보고서 구조가 정상인지 자동 테스트합니다.
3. Staging: 실제 고객 자료와 유사한 샘플을 넣어 결과 품질과 화면을 확인합니다.
4. Production 승인: 고객에게 제공해도 되는 결과인지 최종 검토합니다.
5. 운영 배포: 승인된 버전을 고객 서비스에 반영합니다.

특히 혼자 운영하는 사업에서는 실수 한 번이 고객 신뢰에 영향을 줄 수 있습니다. 이 구조를 표준화하면 모든 고객 프로젝트에서 같은 품질 관리 절차를 반복해서 활용할 수 있습니다.

8. 반드시 기억할 내용

- [ ] Dev, staging, production은 실행 목적이 서로 다릅니다.
- [ ] staging과 production은 같은 코드를 실행하되 환경변수, API 키, 인스턴스 설정 등을 분리합니다.
- [ ] staging 전용 API 키와 최소 권한 원칙을 적용합니다.
- [ ] GitHub Environments의 환경별 변수로 배포 대상을 구분합니다.
- [ ] CI 테스트를 통과한 변경만 staging과 production으로 진행합니다.
- [ ] staging에서 직접 확인한 뒤 main으로 PR을 올립니다.
- [ ] production은 승인자가 승인한 뒤에만 배포합니다.
- [ ] 배포 후 `/version`으로 실제 배포된 커밋을 확인합니다.

한 문장 요약

이번 챕터는 개발·테스트·운영 환경을 분리하고, 자동 테스트와 staging 검증, 운영 승인이라는 품질 게이트를 거쳐 안전하게 AI 서비스를 배포하는 구조를 만드는 과정입니다.

Chapter05

`/강의정리` 챕터 05. PR로만 바뀌는 브랜치

1. 강의 목적

이번 강의의 목적은 GitHub 브랜치 보호 규칙(Branch Protection Rules) 을 설정하여, `main`과 `staging` 브랜치가 정해진 품질 검사를 통과한 뒤에만 변경되도록 만드는 것입니다.

앞선 강의에서 CI 테스트와 배포 승인 절차를 만들었지만, 아직 두 가지 문제가 남아 있었습니다.

- 테스트가 실패해도 머지 버튼을 누를 수 있습니다.
- PR(Pull Request)을 거치지 않고 `git push`로 직접 변경할 수 있습니다.

브랜치 보호 규칙은 이 두 가지 빈틈을 막아 줍니다.

2. 중학생도 이해할 수 있는 설명

회사에서 중요한 문서를 수정한다고 생각해 보겠습니다.

- `main`: 실제 고객에게 제공하는 최종 문서
- `staging`: 최종 공개 전에 확인하는 검토용 문서
- `PR`: 수정 내용을 검토해 달라고 요청하는 절차
- `test`: 수정한 내용에 문제가 없는지 확인하는 검사
- 브랜치 보호 규칙: 정해진 절차를 거치지 않으면 문서를 바꿀 수 없게 하는 규칙

지금까지는 검사에서 문제가 발견되어도 담당자가 그냥 최종 문서를 바꿀 수 있었습니다.

브랜치 보호 규칙을 설정하면 PR을 제출하고 `test` 검사를 통과해야만 변경할 수 있습니다.

이번 실습에서는 혼자 작업하므로 다른 사람의 리뷰 승인은 요구하지 않습니다. 대신 자동 테스트는 반드시 통과하도록 설정합니다.

3. 핵심 개념과 설정

설정 항목	의미	이번 실습의 설정
대상 브랜치	보호할 브랜치	`main`, `staging`
PR 필수	직접 변경하지 못하게 제한	활성화
리뷰 승인 수	다른 사람의 승인 요구	0명
필수 상태 체크	자동 테스트 통과 요구	`test`
관리자에게도 적용	저장소 소유자도 규칙 준수	활성화
다른 브랜치	이번 설정 대상이 아닌 브랜치	변경하지 않음

여기서 중요한 것은 PR을 요구하는 것과 테스트 통과를 요구하는 것은 서로 다른 규칙이라는 점입니다.

PR만 필수로 설정하면 테스트 실패 상태에서도 머지할 수 있습니다. 반대로 테스트만 요구하면 PR 없이 직접 변경하는 경로가 남을 수 있습니다. 두 규칙을 함께 적용해야 합니다.

4. 실습: 브랜치 보호 규칙 설정하기

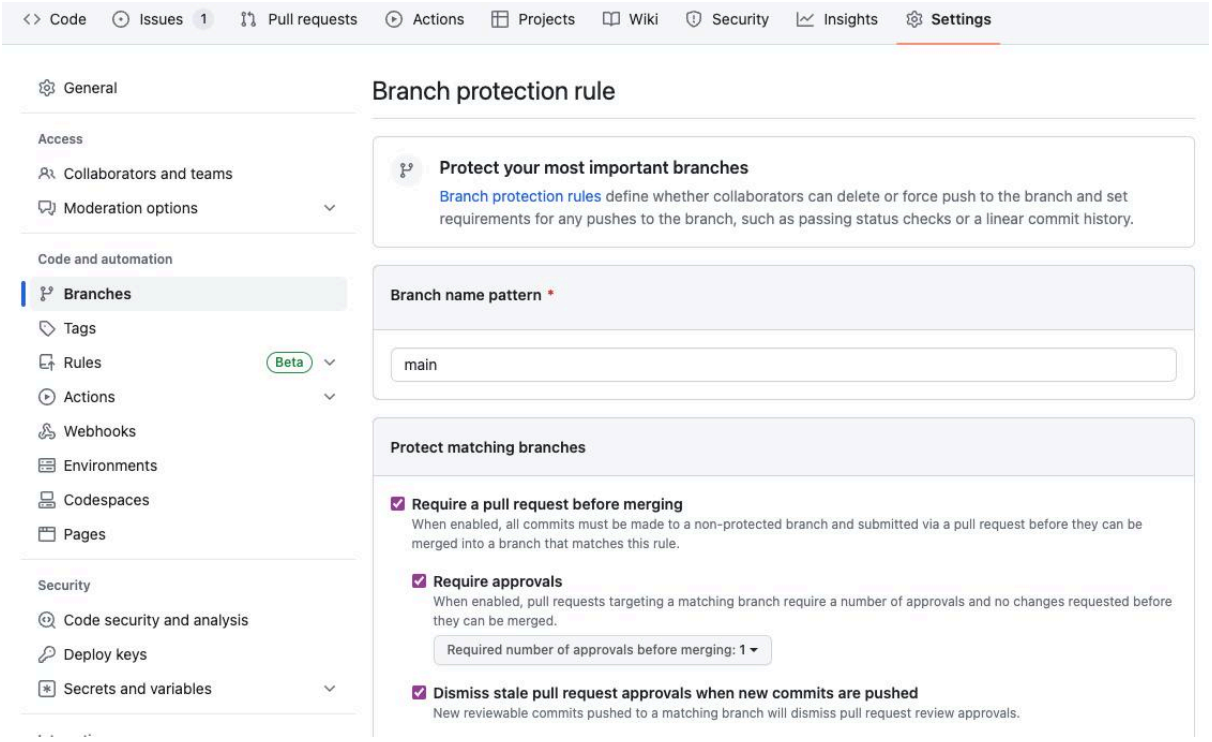
방법 A. 명령어로 설정

저장소 폴더에서 실행합니다. 아래 명령의 `{owner}`와 `{repo}`는 실제 GitHub 사용자 또는 조직 이름과 저장소 이름으로 바꿔야 합니다.

```
gh api -X PUT "repos/{owner}/{repo}/branches/main/protection" --input - <<'EOF'
{
  "required_status_checks": {
    "strict": false,
    "checks": [{"context": "test"}]
  },
  "enforce_admins": true,
  "required_pull_request_reviews": {
    "required_approving_review_count": 0
  },
  "restrictions": null
}
EOF
```

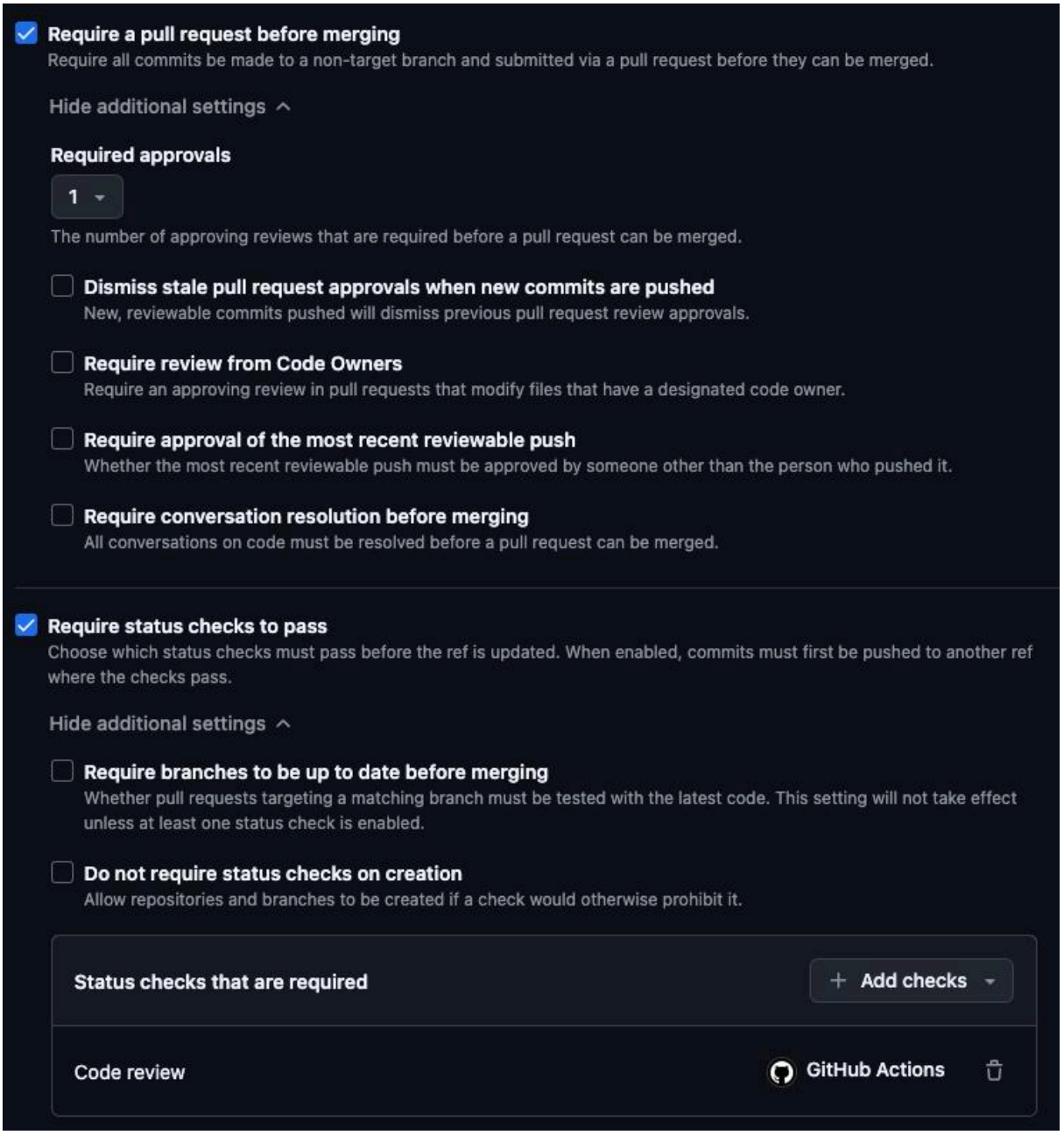
`staging`에도 같은 명령을 실행하되, URL의 `main`만 `staging`으로 바꿉니다.

방법 B. GitHub 화면에서 설정



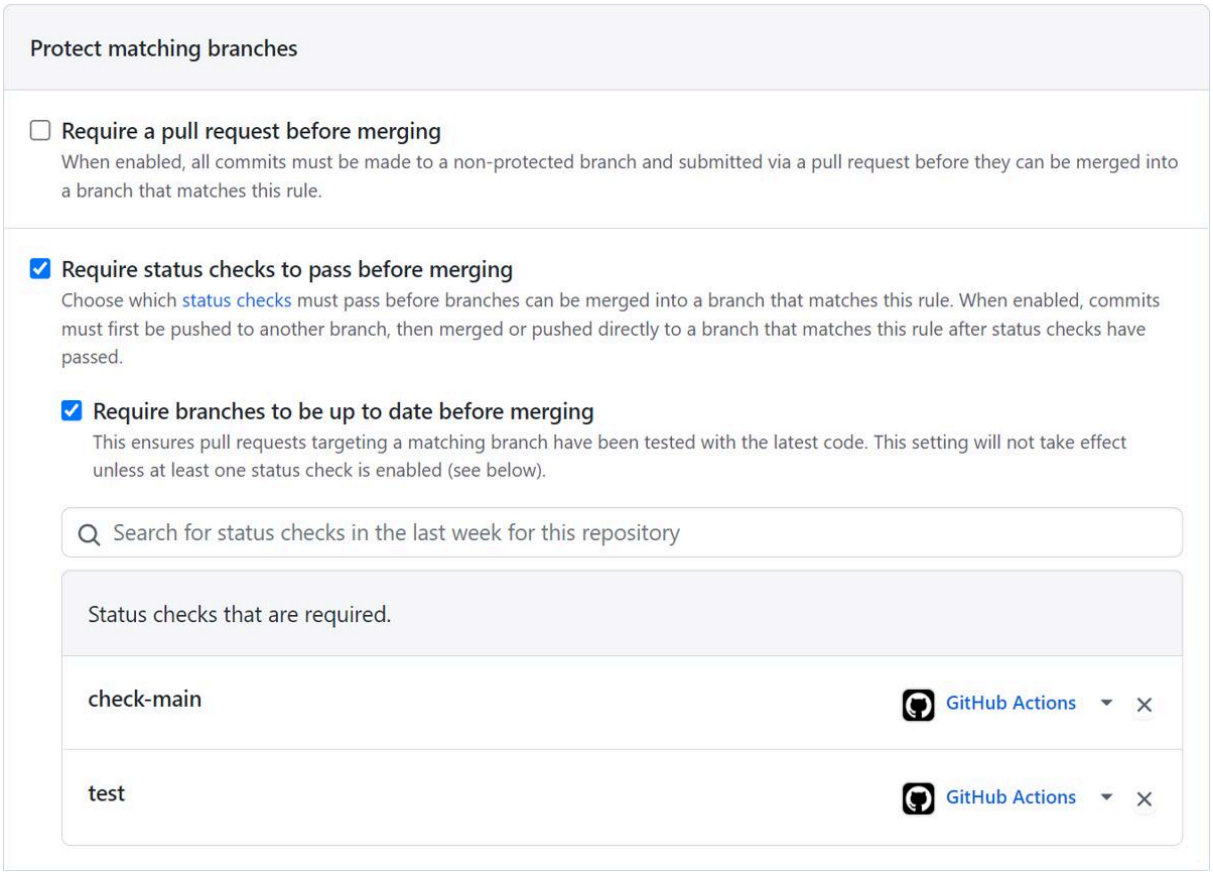
1단계. 브랜치 보호 설정 열기

GitHub 저장소 → Settings → Branches → Add classic branch protection rule로 이동합니다.



2단계. PR 필수 설정

Branch name pattern에 `main`을 입력합니다. `Require a pull request before merging`을 활성화하고, 승인 인원은 0명으로 둡니다.



3단계. 테스트 통과 필수 설정

`Require status checks to pass before merging`을 활성화하고 상태 체크 목록에서 `test`를 선택합니다. 관리자 우회 금지 옵션도 활성화합니다.

4단계. 저장하고 반복하기

규칙을 저장한 뒤 `staging`에도 동일하게 설정합니다.

주의할 점은 상태 체크 목록에 `test`가 보이지 않을 수 있다는 것입니다. 이 경우 앞선 강의에서 설정한 CI 테스트가 먼저 실행되어 GitHub에 상태 체크가 등록되었는지 확인해야 합니다.

5. 설정이 제대로 되었는지 확인하기

다음 항목을 순서대로 확인합니다.

- [ ] `main`에 브랜치 보호 규칙이 설정되어 있습니다.
- [ ] `staging`에도 같은 규칙이 설정되어 있습니다.
- [ ] 두 브랜치 모두 PR을 통한 변경만 허용합니다.
- [ ] 필수 상태 체크에 `test`가 등록되어 있습니다.

- [ ] 저장소 관리자에게도 규칙이 적용됩니다.
- [ ] 다른 브랜치나 저장소 설정은 변경하지 않았습니다.

마지막으로 GitHub의 Settings → Branches 화면에서 두 브랜치의 규칙을 확인합니다.

강의에서는 `main`에서 직접 `git push`를 시도했을 때 보호 규칙에 의해 거절되는지 확인하도록 안내합니다. 실제 확인을 위해 일부러 실패하는 커밋을 만들기보다는, 우선 설정 화면을 확인하는 편이 안전합니다. 직접 푸시 테스트를 진행한다면 작업 중인 변경 사항을 먼저 백업하고, 강의에서 안내한 초기화 명령은 실행 전 대상 브랜치와 삭제될 변경 사항을 반드시 확인해야 합니다.

6. AI 개발자가 알아야 하는 이유

AI가 코드를 작성하거나 코딩 에이전트가 자동으로 파일을 수정하더라도, 그 결과가 항상 올바르다는 보장은 없습니다.

따라서 AI 개발 환경에서는 다음 세 가지가 함께 작동해야 합니다.

1. CI 테스트: 코드에 문제가 있는지 자동 검사합니다.
2. 브랜치 보호: 검사에 실패한 코드를 중요한 브랜치에 반영하지 못하게 합니다.
3. 배포 승인: 운영 환경에 배포하기 전에 사람이 마지막으로 확인합니다.

특히 코딩 에이전트에게 작업을 맡길 때는 단순히 "코드를 수정해 줘"라고 지시하는 것만으로는 부족합니다. 에이전트가 변경할 수 있는 범위와 반드시 통과해야 하는 검증 조건도 함께 설정해야 합니다.

7. 지금까지의 강의와 연결하기

① PR 생성

변경 내용을 검토 요청합니다.

② CI의 `test` 통과

실패하면 머지할 수 없습니다.

③ PR 머지

보호 규칙을 충족한 변경만 반영됩니다.

④ 배포 및 승인

앞서 설정한 staging 검증과 운영 배포 승인 절차로 이어집니다.

이전 챕터에서 만든 CI와 배포 환경에 이번 브랜치 보호 규칙을 더하면, 코드 검사부터 운영 배포까지 단계별로 통제할 수 있습니다.

8. 반드시 기억할 사항

- `main`과 `staging`은 PR로만 변경할 수 있도록 설정합니다.
- 리뷰 승인 수를 0으로 두더라도 `test` 상태 체크는 필수입니다.
- `enforce_admins: true`는 저장소 관리자에게도 보호 규칙을 적용합니다.
- `test`가 상태 체크 목록에 없다면 CI 실행 여부를 먼저 확인합니다.
- 보호 규칙은 설정만 하는 것으로 끝나지 않습니다. 실제 GitHub 화면에서 두 브랜치의 규칙이 올바르게 적용되었는지 확인해야 합니다.

한 문장 요약

브랜치 보호 규칙은 PR 필수와 CI 테스트 통과 조건을 결합해, 관리자라도 검증되지 않은 코드를 `main`이나 `staging`에 직접 반영하지 못하도록 하는 안전장치입니다.

Chapter06

/강의정리 챕터 06. CI 테스트에 걸린 PR 고치기

1. 강의 목적

이번 강의의 핵심은 테스트가 실패한 이유를 GitHub PR 화면과 Actions 로그에서 찾아내고, 자동 테스트가 잡지 못하는 문제는 staging 환경에서 직접 확인하는 것입니다.

앞선 강의에서 CI 테스트, 브랜치 보호, staging·production 배포 환경을 만들었습니다. 이번에는 실제로 문제가 있는 변경을 만들어 품질 관리 장치들이 제대로 작동하는지 확인합니다.

강의는 크게 네 가지 문제를 다룹니다.

- 문제 A~C: CI 테스트가 실패하는 변경을 찾아 원인을 분석합니다.
- 문제 D: CI 테스트는 통과하지만 화면에서 문제가 드러나는 변경을 확인합니다.
- 추가 과제: 테스트를 더 만들거나, 배포 알림을 추가하거나, 자신의 앱에 CI를 적용합니다.

2. 중학생도 이해할 수 있는 설명

학교에서 수행평가 과제를 제출한다고 생각해 보겠습니다.

- CI 테스트: 정답이 맞는지 자동으로 채점하는 시험

- PR 화면: 제출한 답안과 채점 결과를 보는 곳
- Actions 로그: 어느 문제에서 왜 틀렸는지 알려 주는 채점 기록
- staging: 선생님이 실제로 사용해 보고 확인하는 예비 환경
- production: 실제 사용자에게 공개된 운영 환경

자동 채점으로 확인할 수 있는 오류는 CI가 잡아냅니다. 하지만 버튼이 화면에서 사라지거나 AI 답변의 말투가 어색한 문제는 자동 테스트가 확인하도록 만들어 두지 않았다면 놓칠 수 있습니다.

따라서 자동 테스트와 사람이 직접 사용해 보는 검증은 서로 보완하는 관계입니다.

3. 문제 A~C: CI 실패 원인 찾기

이번 실습에서는 일부러 문제가 생기도록 만든 패치 파일을 적용합니다. 중요한 것은 코드를 추측해서 고치는 것이 아니라, 실패한 테스트와 실제 변경된 코드를 연결해 원인을 설명하는 것입니다.

공통 작업 순서

1. `staging` 브랜치를 최신 상태로 받고, 문제별 작업 브랜치를 만듭니다.
2. `starter/incidents/` 안의 패치 파일을 적용하고 커밋한 뒤 GitHub에 푸시합니다.
3. `staging`을 대상으로 PR을 만들고 `gh pr checks -watch`로 검사 결과를 확인합니다.
4. PR 화면과 Actions 로그에서 실패한 테스트 이름과 실패 메시지를 찾습니다.
5. 실패한 테스트가 확인하는 동작과 `Files changed`에 표시된 코드를 연결해 원인을 한 문장으로 정리합니다.
6. 머지가 차단되는지 확인한 후 PR을 닫고 작업 브랜치를 정리합니다.

세 가지 문제의 핵심

문제	변경 내용	조사할 핵심
A. `safety-exact-match`	위기 표현 판정 방식 변경	위기 표현을 판정하는 조건이 기존 기대 동작을 깨뜨렸는지
B. `weather-keyword`	날씨 도구 선택 키워드 변경	어떤 날씨 입력에서 도구 선택이 기대와 달라졌는지
C. `reminder-timezone`	약속 시각에서 시간대 제거	시간대 변환이 빠져 약속 시각이 달라졌는지

강의 자료에는 각 문제의 실제 실패 테스트 이름과 로그 메시지가 제시되어 있지 않습니다. 따라서 정확한 실패 원인은 직접 실행한 결과를 근거로 확인해야 합니다.

기본 실행 명령

아래에서 `<패치 이름>`은 실제 패치 파일 이름으로 바꿔야 합니다.

```
git switch staging
git pull
git switch -c broken-<패치 이름>
git apply starter/incidents/<패치 이름>.patch
git commit -am "<패치 이름> 확인용 변경"
git push -u origin broken-<패치 이름>

gh pr create --base staging \
  --title "<패치 이름> 확인용 변경" \
  --body "CI 결과 확인용. 머지하지 않는다"

gh pr checks --watch
gh run list --limit 3
gh run view <실행 번호> --log-failed
```

원인을 기록한 다음 PR과 브랜치를 정리합니다.

```
gh pr close --delete-branch
git switch staging
git branch -D broken-<패치 이름>
```

주의: `git branch -D`는 로컬 브랜치를 강제로 삭제합니다. 삭제할 브랜치가 맞는지 확인하고 실행하세요. 문제 A~C는 원인 분석 실습이므로 수정하거나 머지하지 않습니다.

4. 문제 D: 테스트를 통과했는데 버튼이 사라진다면?

문제 D는 앞의 세 문제와 성격이 다릅니다.

화면 파일 `static/index.html`에서 보내기 버튼의 스타일 `.send{...}` 안에 `display:none;`을 넣으면 버튼이 보이지 않게 됩니다.

그런데 자동 테스트가 화면에 버튼이 보이는지 검사하지 않는다면 CI는 통과할 수 있습니다.

문제 D의 품질 검증 흐름

1. CI 테스트 통과

현재 테스트는 버튼의 표시 여부를 검사하지 않습니다.

2. staging에서 직접 확인

화면을 열어 보내기 버튼이 사라진 것을 발견합니다.

3. Revert PR로 되돌리기

문제 변경을 취소하고 staging에서 버튼이 돌아왔는지 확인합니다.

실습에서 특히 지켜야 할 조건은 다음과 같습니다.

- 작업 브랜치는 `staging`에서 생성합니다.
- PR 대상은 `staging`입니다.
- CI 결과를 확인한 후에만 머지합니다.
- staging 화면에서 버튼이 사라졌는지 확인합니다.
- 문제가 확인되면 Revert PR을 만들어 머지합니다.
- 이 변경은 `main`으로 보내지 않습니다.

코딩 에이전트에게 맡길 때도 머지 권한을 분명하게 구분해야 합니다. 강의 지시처럼 PR 생성과 상태 체크 확인까지만 맡기고, 실제 머지는 직접 결정하도록 할 수 있습니다.

5. 주관식 퀴즈 모범 답안

Q1. 보내기 버튼이 사라진 변경을 CI에서 미리 막으려면 무엇이 필요할까요?

화면에 보내기 버튼이 표시되는지 확인하는 자동 테스트가 필요합니다. 예를 들어 브라우저 테스트를 추가해 버튼이 화면에 존재하고 사용자에게 보이는지 검사하도록 만들 수 있습니다. 이 테스트를 CI에서 실행하면 버튼을 숨기는 변경이 PR 단계에서 실패하므로 staging에 도달하기 전에 막을 수 있습니다.

Q2. 문제 A~C 중 하나를 골라 원인과 단서를 설명하세요.

문제 C를 선택한 예시입니다. 실제 테스트 이름과 실패 메시지는 실행 결과에 맞게 보완해야 합니다.

문제 C인 `reminder-timezone`을 선택했습니다. 약속 시각을 처리할 때 시간대 정보가 제거되어 기대한 시각과 실제 저장 또는 처리되는 시각이 달라지는 문제가 발생할 수 있습니다. 처음에는 PR의 CI 상태 체크에서 실패를 확인하고, Actions의 실패 로그에서 관련 테스트와 메시지를 찾은 다음 `Files changed`에서 시간대 처리가 변경된 줄을 확인합니다. 로컬에서도 동일한 테스트를 실행했다면 같은 오류를 재현할 수 있으므로 잡을 수 있었겠지만, 로컬에서 테스트를 실행하지 않았다면 미리 발견하지 못했을 수 있습니다.

6. 선택 실습 세 가지

기본 — 예시를 테스트 케이스로 추가

`docs/examples.md`에서 판정 방법이 `코드`인 예시를 하나 골라 `tests/`에 테스트를 추가합니다. 페이크 LLM을 사용하고 테스트 위에 예시 번호와 기대 동작을 주석으로 기록합니다.

완료 기준: 로컬 `pytest`와 CI에서 통과하며, 동작을 일부러 깨뜨리면 테스트가 실패합니다.

중간 — 배포 결과를 메신저로 알리기

배포 성공 또는 실패, 환경 이름, 커밋 SHA를 Slack이나 Discord로 알립니다. 앞 단계가 실패하더라도 알림 단계는 실행되도록 구성합니다.

완료 기준: 성공과 실패 모두 알림이 오며, 웹훅 주소가 워크플로 파일에 직접 노출되지 않습니다.

도전 — 자신의 앱에 CI 적용

자신의 앱에서 입력과 기대 동작의 예시를 하나 정하고, 이를 검사하는 테스트와 GitHub Actions 워크플로를 추가합니다.

완료 기준: PR을 올리면 테스트가 실행되고, 해당 동작을 일부러 망가뜨리면 CI가 실패합니다.

중간 과제에서 웹훅 주소는 비밀번호처럼 취급해야 합니다. `gh secret set DEPLOY_WEBHOOK_URL`로 GitHub 저장소 시크릿에 저장하고 워크플로에서는 시크릿 이름으로만 읽습니다. 주소 자체를 코드나 로그에 노출하지 않아야 합니다.

7. AI 학습 및 1인 창업과의 연결

이번 강의는 AI 서비스 개발뿐 아니라 앞으로 컨설팅 사업에서 고객의 AI 서비스를 점검할 때도 중요한 관점을 제공합니다.

- 테스트 가능한 요구사항 만들기: “잘 작동해야 한다”가 아니라 “이 입력에는 이 결과가 나와야 한다”로 구체화합니다.
- 자동화의 한계 이해하기: 테스트가 통과했다고 모든 품질이 보장되는 것은 아닙니다.
- 검증 절차 설계하기: 자동 검사, staging 실사용 검증, 운영 배포 전 확인을 구분합니다.
- AI 에이전트 통제하기: 에이전트에게 수정과 PR 생성을 맡기더라도, 검증 기준과 머지 권한은 명확히 구분합니다.

예를 들어 My-Story-Maker(MSM)에서는 입력한 글이 원문의 의미를 보존하는지, Travel Creator에서는 일정 변경 후 시간과 순서가 올바른지 자동 테스트로 확인할 수 있습니다. 반면 결과물의 자연스러움이나 실제 사용 편의성은 사람이 직접 확인하는 절차가 추가로 필요합니다.

8. 반드시 기억할 사항

1. 실패한 CI 로그는 원인을 찾기 위한 증거입니다. 테스트 이름, 실패 메시지, 변경된 코드를 함께 확인합니다.
2. CI 테스트는 작성된 검사만 수행합니다. 검사하지 않는 문제는 놓칠 수 있습니다.
3. staging은 실제 사용자에게 공개하기 전에 문제를 발견하는 환경입니다.
4. 문제 D는 `main`으로 보내지 않습니다. staging에서 확인하고 Revert PR로 되돌립니다.
5. 자동 테스트와 사람의 확인은 대체 관계가 아니라 보완 관계입니다.

한 문장 요약

좋은 품질 관리란 CI 테스트로 코드 수준의 오류를 자동으로 잡고, staging에서 실제 사용 문제를 확인하며, 검증된 변경만 운영 환경에 배포하는 체계를 만드는 것입니다.

Chapter07

`/강의정리` 챕터 07. 오늘 배운 것 — 품질 게이트 회고

1. 강의 목적

오늘 강의의 핵심은 AI 서비스의 오류를 미리 발견하고, 검증되지 않은 코드가 운영 환경에 배포되지 않도록 품질 관리 체계를 만드는 것입니다.

테스트를 작성하는 것에서 끝나지 않고, CI 테스트, 환경 분리, 브랜치 보호, staging 검증, 운영 배포 승인을 연결해 실제로 작동하는 안전장치를 만들었습니다.

2. 중학생도 이해할 수 있는 설명

자동차를 출고한다고 생각해 보겠습니다.

- CI 테스트: 자동차의 브레이크와 엔진을 자동으로 검사하는 시험
- 환경 분리: 개발용 시험장, 출고 전 검수장, 실제 도로를 구분하는 것
- 브랜치 보호: 검사를 통과하지 않은 자동차는 다음 단계로 넘어가지 못하게 하는 규칙
- staging 검증: 실제로 운전해 보며 자동 검사에서 발견하지 못한 문제를 찾는 과정
- production 승인: 모든 검증을 마친 뒤 실제 고객에게 공개하도록 최종 승인하는 절차

자동 검사만으로 모든 문제를 발견할 수는 없습니다. 그래서 자동 검사와 실제 사용 확인을 함께 운영해야 합니다.

3. 오늘 배운 핵심 내용

① CI 테스트: 자동으로 오류 찾기

실제 LLM API 대신 페이크 LLM을 사용하면 API 비용 없이 테스트를 반복할 수 있고, 매번 같은 결과를 얻을 수 있습니다.

약속 시각 테스트에서는 사용자가 기대하는 오전 8시가 올바르게 처리되는지 확인했습니다. 이 테스트를 통해 시간대 정보가 빠진 버그를 발견할 수 있었습니다.

이후 테스트를 `main`에 올리고 GitHub Actions를 설정해 PR마다 테스트가 실행되도록 했습니다. 서버와 CI 러너가 UTC 환경에서 실행되기 때문에 로컬에서는 드러나지 않던 시간대 오류도 확인할 수 있었습니다.

② 환경 분리: 같은 코드, 다른 설정

환경	주요 목적	관리할 항목
dev	개발 및 실험	개발용 설정과 키
staging	배포 전 검증	테스트용 서버와 시크릿, 화면 표시
production	실제 사용자 서비스	운영용 서버와 시크릿, 인스턴스 설정

같은 코드라도 환경에 따라 서버 이름, API 키, 인스턴스 수, 화면 표시 등을 다르게 설정할 수 있습니다.

또한 Secret Manager에 환경별 시크릿을 분리하고, 각 런타임 서비스 계정에는 필요한 시크릿만 읽을 권한을 부여했습니다. 이는 보안을 지키면서 서비스를 운영하기 위한 방법입니다.

③ 실패에서 배운 것: 자동 테스트에도 한계가 있다

오늘 확인한 문제는 두 가지 유형으로 나뉩니다.

- 시간대 오류: 로컬에서는 정상처럼 보였지만 UTC 환경에서는 9시간 차이가 발생했습니다. 약속 시각 테스트와 CI가 이 문제를 발견하는 데 도움이 됐습니다.
- 보내기 버튼 오류: 테스트가 모두 통과했지만 버튼이 사라졌습니다. 화면을 직접 확인하는 staging 검증이 필요했습니다.

즉, 테스트가 통과했다는 것은 작성된 테스트가 확인하는 범위에서 문제가 발견되지 않았다는 뜻이지, 모든 기능이 완벽하다는 뜻은 아닙니다.

④ 품질 게이트: 검증되지 않은 변경 차단하기

브랜치 보호 규칙으로 `main`과 `staging`에 PR을 거치지 않고 직접 변경하는 경로를 막고, 필수 `test` 상태 체크가 실패하면 머지하지 못하도록 설정했습니다.

production 배포에서는 에이전트에게 최종 승인을 맡기지 않고, staging의 커밋과 비교한 뒤 직접 승인하도록 했습니다.

4. 주관식 퀴즈 모범 답안

질문은 아침에 생각했던 약속 시각 변경 방지 방법을 다시 살펴보고, 오늘 실제로 만든 품질 게이트와 만들지 않은 지점을 구분하는 것입니다.

실제로 만든 품질 게이트: 약속 시각 테스트를 추가해 사용자가 보는 오전 8시가 올바르게 처리되는지 확인했습니다. GitHub Actions에서 PR마다 테스트를 실행하고, `main`과 `staging`의 브랜치 보호 규칙을 설정해 필수 테스트가 실패하면 머지하지 못하도록 했습니다. staging에서는 화면을 직접 확인하고, production 배포 전에는 커밋을 비교한 뒤 직접 승인하도록 했습니다.

오늘 만들지 않은 지점: 약속 시각 테스트는 특정 동작을 검사하지만, 모든 시간대 관련 상황을 검사하는 것은 아닙니다. 다양한 시간대와 날짜 경계 조건을 추가로 테스트하거나, 실제 사용자 화면의 동작을 자동으로 확인하는 테스트는 별도로 확장할 수 있습니다. 다만 이러한 추가 테스트는 오늘 강의에서 실제로 만들었다고 제시된 내용은 아닙니다.

5. AI 개발자와 1인 창업에 연결하기

앞으로 AI 컨설팅 사업을 운영하거나 고객의 AI 서비스를 점검할 때도 오늘 배운 내용을 활용할 수 있습니다.

1. 고객의 요구사항을 테스트할 수 있는 구체적인 조건으로 바꿉니다.
2. 자동 테스트로 반복 검증할 수 있는 문제를 구분합니다.
3. 자동 테스트만으로 확인할 수 없는 문제는 staging에서 실제로 사용해 봅니다.
4. 브랜치 보호와 배포 승인으로 검증되지 않은 변경이 운영 환경에 들어가는 것을 방지합니다.

특히 코딩 에이전트에게 개발을 맡길 때는 코드를 작성하는 능력만큼이나 에이전트가 만든 결과물을 어떻게 검증하고 통제할 것인지가 중요합니다.

6. 반드시 기억할 사항

- 페이크 LLM은 비용과 결과의 변동성을 줄여 테스트하기 좋습니다.
- 로컬과 서버의 환경 차이는 시간대 같은 오류를 일으킬 수 있습니다.

- CI는 자동화된 검사이며, 검사하지 않는 화면 문제까지 보장하지는 않습니다.
- staging은 자동 테스트를 통과한 변경을 실제 사용 관점에서 검증하는 환경입니다.
- 브랜치 보호와 운영 배포 승인은 품질 관리 절차가 우회되지 않도록 돕습니다.

한 문장 요약

좋은 AI 서비스 운영은 테스트를 작성하는 데서 끝나지 않고, CI·환경 분리·브랜치 보호·staging 검증·운영 승인을 연결해 오류를 일찍 발견하고 안전하게 배포하는 체계를 만드는 것입니다.